Firebase Google Cloud React 19 React Native / Expo LangGraph + AI Paddle

Pocket — System Design Document

Read-later web app with AI summaries, text-to-speech, and reading streaks

Version 1.3 | March 2026 | Project ID: `finn-2c4c5` | Region: `us-central1`

CONTENTS

1. System Overview

Pocket is a full-stack read-later application built on the Firebase ecosystem. Users save articles from the web, which are server-side fetched, parsed, and stored in Firestore. Premium features include AI-powered summaries (Groq LLM via LangGraph), AI text-to-speech (HuggingFace Kokoro), highlights, collections, and reading streak gamification.

9	5	15	2	3
CLOUD FUNCTIONS	FIRESTORE COLLECTIONS	CLIENT ROUTES	LANGGRAPH PIPELINES	CLIENT PLATFORMS

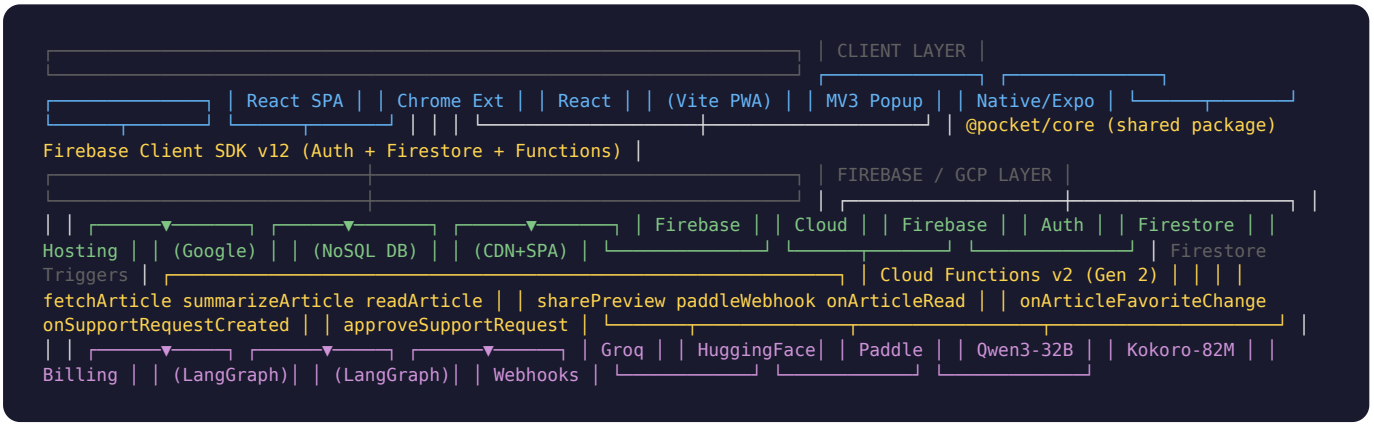
TECH STACK

⚛️ React 19 · 📱 React Native / Expo 52 · ⚡ Vite 7 · 🔥 Firebase v12 · ☁️ Cloud Functions v2 · 🤖 LangGraph · 🧠 Groq (Qwen3-32B) · 🔊 Kokoro TTS · 💳 Paddle · 🧩 Chrome MV3

MONOREPO STRUCTURE

```
pocket/                    # npm workspaces + Turborepo
├── src/                   # React web app (Vite + React 19)
├── functions/             # Firebase Cloud Functions (Node 20, CommonJS)
├── extension/             # Chrome Extension (Manifest V3)
├── apps/
│   └── pocket-native/     # React Native / Expo app (iOS + Android)
├── packages/
│   └── core/              # @pocket/core — shared firebase, context, utils
├── docs/                  # Business analysis + system design PDFs
└── scripts/               # Build + diagnostic scripts
```

2. High-Level Architecture



3. Frontend Architecture (Web)

3.1 Project Structure

```
src/
├── App.jsx                # Route definitions + auth guards
├── components/            # Layout, ArticleCard, EditorPicks, StreakBadge, etc.
├── pages/                 # MyList, Reader, Archive, Favorites, Tags, Premium, etc.
├── context/               # AuthContext, ThemeContext
├── hooks/                 # usePremium, useScrollRestore
├── firebase/              # config.js, auth.js, articles.js
└── utils/                 # articleFetcher.js (Cloud Function wrappers)
```

3.2 Route Map

PATH	COMPONENT	AUTH	LAYOUT
`/`	Landing (guest) / MyList (user)	Dynamic	Conditional
`/archive`	Archive	Required	Yes
`/favorites`	Favorites	Required	Yes
`/tags` , `/tags/:tag`	Tags	Required	Yes
`/collections`	Collections (premium)	Required	Yes
`/read/:id`	Reader (fullscreen)	Required	No
`/save?url=...`	SaveHandler	Required	No
`/premium`	Premium	Required	Yes
`/support`	Support	Required	Yes
`/about`	About (release notes)	Required	Yes
`/p/:code`	ShareRedirect	No	No
`/blog/reading-toolkit`	BlogReadingToolkit	No	No
`/terms` , `/privacy` , `/refund`	Legal pages	No	No
`/login`	Login (Google OAuth)	Guest only	No

3.3 State Management

```
ThemeProvider ← localStorage ('theme': 'light' | 'dark') | AuthProvider ← Firebase Auth + Firestore user profile | ├── user: Firebase User (uid, email, displayName) | ├── userProfile: /users/{uid} doc (isPremium, streaks, badges) | ├── isPremium: boolean (derived) | └── refreshProfile(): refetch from Firestore | └── Component Tree ├── useAuth() → user + profile + premium ├── usePremium() → isPremium shorthand └── useTheme() → theme + toggle()
```

3.4 Build Configuration

SETTING	VALUE
Bundler	Vite 7 + React plugin
Output	`dist/` (SPA fallback to `index.html`)
Code splitting	Manual chunks: `firebase`, `readability`, `router`, `vendor`
Bundle size	~328 KB app + ~359 KB firebase (gzip: ~103 KB + ~112 KB)
Styling	CSS Modules (`*.module.css`)
PWA	Service worker + `manifest.webmanifest`

4. React Native / Expo App

Native iOS and Android app built with Expo SDK 52 and Expo Router. Shares Firebase backend and business logic via `@pocket/core` package. Currently scaffolded and in active development.

4.1 Project Structure

```
apps/pocket-native/
├── app.config.js              # Expo config (icons, splash, bundle IDs)
├── package.json               # @pocket/native — Expo 52, React Native 0.76.9
├── eas.json                   # EAS Build profiles (dev, preview, production)
├── metro.config.js            # Metro bundler config (monorepo support)
├── babel.config.js            # Babel with expo preset
│
├── app/                       # Expo Router file-based routing
│   ├── _layout.jsx            # Root layout (auth gate)
│   ├── (auth)/
│   │   ├── _layout.jsx        # Auth layout
│   │   └── login.jsx          # Login screen (Google OAuth)
│   ├── (tabs)/
│   │   ├── _layout.jsx        # Bottom tab navigator
│   │   ├── index.jsx          # My List tab
│   │   ├── favorites.jsx      # Favorites tab
│   │   ├── archive.jsx        # Archive tab
│   │   └── settings.jsx       # Settings tab
│   └── read/
│       └── [id].jsx           # Article reader screen
│
├── components/
│   └── ArticleCard.jsx        # Native article list item
│
└── src/
    └── firebase/
        └── setup.js           # Firebase initialization for native
```

4.2 Navigation Architecture

```
Root Layout (app/_layout.jsx)
├── Not authenticated ──► (auth)/login.jsx
│   Google OAuth via expo-web-browser
└── Authenticated ──► (tabs)/_layout.jsx  Bottom Tab Navigator
    ├── My List (index.jsx) — Bookmark icon
    ├── Favorites (favorites.jsx) — Heart icon
    ├── Archive (archive.jsx) — Archive icon
    └── Settings (settings.jsx) — Settings icon
    └── Push to: read/[id].jsx — Article reader
```

4.3 Dependencies & Configuration

PACKAGE	VERSION	PURPOSE
`expo`	~52.0.46	Expo SDK runtime
`react-native`	0.76.9	React Native framework
`expo-router`	~4.0.17	File-based routing
`@pocket/core`	workspace:*	Shared Firebase, context, utils
`firebase`	^12.9.0	Same Firebase SDK as web

PACKAGE	VERSION	PURPOSE
`expo-share-intent`	^3.1.1	iOS/Android share sheet integration
`expo-web-browser`	~14.0.2	OAuth redirect handling
`lucide-react-native`	^0.475.0	Native icon library
`react-native-webview`	13.12.5	Article content rendering

4.4 Share Intent Integration

> **iOS:** Share Extension with `NSExtensionActivationSupportsWebURLWithMaxCount: 1` — shows Pocket in the iOS share sheet for web URLs.
> **Android:** Intent filter for `android.intent.action.SEND` with `text/plain` — receives shared URLs from any app.

4.5 Build & Deploy (EAS)

```
cd apps/pocket-native
npx expo start              # Dev server
eas build --platform ios    # Cloud build for iOS
eas build --platform android # Cloud build for Android
eas submit --platform ios   # Submit to App Store
eas submit --platform android # Submit to Play Store
```

CONFIG	VALUE
iOS Bundle ID	`com.pocket.app`
Android Package	`com.pocket.app`
Splash Background	`#0A0A0F`
Adaptive Icon BG	`#EF4056` (Pocket red)
Orientation	Portrait only
Dark mode	Automatic (system)

5. Shared Package (@pocket/core)

The `@pocket/core` workspace package provides shared code between the web app and the React Native app. Avoids duplicating Firebase config, auth logic, and article CRUD across platforms.

```
packages/core/
├── package.json               # @pocket/core, ES Modules
└── src/
    ├── firebase/
    │   ├── index.js           # Re-exports config, auth, articles
    │   ├── config.js          # Firebase app init (project finn-2c4c5)
    │   ├── auth.js            # login, logout, onAuthChange
    │   └── articles.js        # getArticles, addArticle, updateArticle, etc.
    ├── context/
    │   ├── index.js           # Re-exports AuthContext, ThemeContext
    │   ├── AuthContext.jsx     # User + profile state
    │   └── ThemeContext.jsx    # Light/dark theme toggle
    └── utils/
        ├── index.js           # Re-exports articleFetcher
        └── articleFetcher.js  # Cloud Function callable wrappers
```

EXPORTS MAP

IMPORT PATH	CONTENTS
`@pocket/core/firebase`	db, auth, functions, googleProvider, getUserProfile, getArticles, addArticle, updateArticle, etc.
`@pocket/core/context`	AuthProvider, useAuth, ThemeProvider, useTheme
`@pocket/core/utils`	fetchAndParse, fetchMetadataOnly, summarizeArticle, readArticleAloud

PEER DEPENDENCIES

```
firebase: ^12.0.0
react: ^19.0.0
react-router-dom: ^7.0.0
```

6. Chrome Extension (MV3)

Manifest V3 Chrome extension that lets users save the current browser tab to their Pocket list with one click.

```
extension/
├── manifest.json              # MV3: permissions, host_permissions, action
├── popup.html                 # Extension popup UI
├── popup.css                  # Popup styles
├── src/popup.js               # Source (React-like UI)
├── popup.bundle.js            # Built IIFE bundle (vite build:ext)
└── icons/
    ├── icon16.png
    ├── icon48.png
    ├── icon128.png
    └── pocket.svg
```

6.1 Manifest Configuration

FIELD	VALUE
manifest_version	3
Permissions	`activeTab` , `tabs`
Host Permissions	`*.googleapis.com` , `*.firebaseapp.com` , `firestore.googleapis.com` , `identitytoolkit.googleapis.com` , `securetoken.googleapis.com`
Action	Default popup: `popup.html` , title: "Save to Pocket"

6.2 Flow

```
User clicks extension icon │ ├── popup.html loads popup.bundle.js ├── Checks onAuthStateChanged() │ ├── Signed in: │
├── Shows user avatar + current tab info │ ├── Tag input for categorization │ ├── "Save to Pocket" button │ │ └──
addArticle() → Firestore │ │ └── fetchArticle fires in background (Reader) │ └── Auto-closes after 900ms on success │
└── Not signed in: └── "Sign in with Google" → OAuth popup (extension origin must be in Firebase authorized domains)
```

6.3 Build

```
npm run build:ext
# → vite build --config vite.config.extension.js
# → extension/src/popup.js → extension/popup.bundle.js (IIFE)
# → Load in chrome://extensions as unpacked extension
```

7. Backend — Cloud Functions

All Cloud Functions are in `functions/index.js` (CommonJS, Node 20). They run on Cloud Functions v2 (Gen 2) in `us-central1`.

FUNCTION	TRIGGER	MEMORY	TIMEOUT	SECRETS	PURPOSE
`fetchArticle`	onCall	512 MiB	30s	—	Server-side article fetcher. JSDOM + Readability + Wayback fallback. Handles Twitter/oEmbed, YouTube transcripts.
`summarizeArticle`	onCall	512 MiB	60s	GROQ_API_KEY	LangGraph: assess → summarize (Groq Qwen3-32B). Returns summary, key points, suggested tags.
`readArticle`	onCall	512 MiB	120s	HF_API_KEY	LangGraph: prepare chunks → synthesize via Kokoro TTS. Returns base64 audio.
`sharePreview`	onRequest	256 MiB	15s	—	OG meta server for `/p/{code}` share links. Returns HTML with OG tags + redirect.
`paddleWebhook`	onRequest	256 MiB	15s	PADDLE_WEBHOOK_SECRET	Paddle Billing webhook. HMAC-SHA256 verification. Sets/revokes `isPremium`.
`onArticleFavoriteChange`	Firestore	256 MiB	—	—	Maintains `/articleStats/{urlHash}` favorite counts for Featured Article.
`onArticleRead`	Firestore	256 MiB	—	—	Updates reading streak, total articles read, awards badges on `isRead` flip.
`onSupportRequestCreated`	Firestore	256 MiB	—	GMAIL_APP_PASSWORD	Sends admin email notification on new support ticket.
`approveSupportRequest`	onCall	256 MiB	—	GITHUB_TOKEN	Admin-only. Creates GitHub issue with `claude-task` label.

7.1 Article Fetch Pipeline

```
Client ──► fetchArticle Cloud Function
  ├── Is Twitter/X? ──► oEmbed API ──► sanitize HTML ──► return
  ├── Is YouTube? ──► extract ytInitialPlayerResponse
  │     ├── title, thumbnail, description
  │     └── fetch captions (JSON3) ──► parse transcript
  └── Regular URL
        ├── fetch(url) ──► HTTP 200? ──► parse
        ├── HTTP 403/429/503? ──► Wayback Machine fallback
        └── Parse HTML
              ├── JSDOM + Readability (primary)
              ├── 14 fallback CSS selectors if Readability returns null
              ├── OG meta extraction (title, image, description)
              └── sanitize-html (whitelist tags + safe CSS)
```

7.2 Known Failure Modes

FAILURE	CAUSE	MITIGATION
JavaScript-rendered SPA	JSDOM doesn't execute JS (React/Next.js)	OG meta returned; content empty. Future: Puppeteer
Paywall / login wall	HTTP 403 or JS-gated	Wayback Machine fallback
Bot blocking	Cloud IPs flagged (Cloudflare, Akamai)	Custom User-Agent + Wayback
Readability null	Non-standard HTML	14 fallback CSS selectors tried in order

8. Data Model — Firestore

```
Firestore Database
├── users/{uid} ← user profile + streak data
│   uid, displayName, email, photoURL, createdAt
│   isPremium, premiumSince, premiumEndedAt
│   paddleCustomerId, paddleSubscriptionId
│   currentStreak, longestStreak, lastReadDate
│   totalArticlesRead, badges[]
│   └── articles/{articleId} ← saved articles
│       url, title, excerpt, heroImage, domain, content
│       wordCount, estimatedReadTime, authors[], tags[]
│       fetchStatus, fetchError
│       isRead, isFavorite, isArchived, readAt, savedAt
│       aiSummary, aiKeyPoints[], aiSuggestedTags[]
│       isVideo, videoId, transcript[]
│       └── highlights/{highlightId} ← text highlights (premium)
│           text, color, note, startOffset, endOffset, createdAt
├── shares/{code} ← Bitly-style share links
│   url, title, heroImage, excerpt, domain, createdAt
├── articleStats/{urlHash} ← global favorite counts
│   url, title, heroImage, excerpt, domain, favoriteCount, lastUpdated
├── supportRequests/{requestId} ← support tickets
│   userId, userName, userEmail, title, description
│   status, createdAt, githubIssueUrl, githubIssueNumber
└── errors/{autoId} ← function error logs
    type, uid, articleId, error, timestamp
```

8.1 Security Rules (Summary)

COLLECTION	READ	WRITE	SPECIAL RULES
`users/{uid}`	Owner only	Owner only	Cannot self-grant `isPremium`
`articles`	Owner only	Owner only	Full CRUD for subcollection owner
`highlights`	Owner only	Owner only	Premium feature
`shares`	Anyone	Auth create only	No update/delete (immutable)
`articleStats`	Auth only	Cloud Function only	Admin SDK bypasses rules
`supportRequests`	Owner + admin	Create: owner; Update: admin	Composite index: userId + createdAt

9. AI Pipeline — LangGraph

9.1 Summarization Pipeline

```
summarizeArticle (Groq Qwen3-32B, max 4000 tokens) START ──► assess (count words) │ ├── < 50 words? ──► END (skip) │ ▼
summarize (LLM call) │ System: "Reply with ONLY valid JSON" │ Strips <think> blocks (Qwen3 thinking mode) │ Persists:
aiSummary, aiKeyPoints, aiSuggestedTags ▼ END → { summary, keyPoints[], suggestedTags[] }
```

9.2 Text-to-Speech Pipeline

```
readArticle (HuggingFace Kokoro-82M, FLAC output) START ──► prepare (strip HTML → sentence-split → ~400 char chunks) │
(cap at ~3000 chars total = ~2-3 min speech) │ ├── < 10 chars? ──► END (skip) │ ▼ synthesize (HuggingFace API per
chunk) │ POST api-inference.huggingface.co/models/hexgrad/Kokoro-82M │ → base64 encode each audio buffer ▼ END → {
audioChunks: [base64...], contentType: "audio/flac" }
```

10. Authentication & Authorization

ROLE	ACCESS
Guest	Landing page, login, blog, legal pages, share redirects
Free user	500 articles, browser TTS, tags, PWA, streaks/badges
Premium user	+ AI TTS, AI summaries, highlights, collections, unlimited saves
Admin	Approve/reject support → creates GitHub issues

11. Payment System — Paddle

```
Premium Page → Paddle.Checkout.open({ customData: { user_id } }) │ ├── User completes payment on Paddle hosted checkout
│ └── Paddle webhook ──► paddleWebhook Cloud Function ├── Verify HMAC-SHA256 (crypto.timingSafeEqual) ├──
subscription.activated → isPremium: true └── subscription.canceled → isPremium: false
```

PLAN	PRICE	NET (AFTER FEES)
Free	$0	—
Premium Monthly	$3.99/mo	~$3.23/mo
Premium Annual	$29.99/yr ($2.50/mo)	~$27.54/yr

12. CI/CD & Deployment

```
git push to master or claude/* | ├── deploy.yml (always runs) | ├── npm ci + npm run build | ├── Deploy Firestore rules + indexes | └── Deploy to Firebase Hosting (dist/ → CDN) | └── deploy-functions.yml (only if functions/** changed) ├── npm ci (functions/) ├── Sync secrets → GCP Secret Manager └── Deploy each function individually: ├── HTTP triggers: gcloud functions deploy (Gen 2) └── Firestore triggers: firebase deploy (Eventarc)
```

GITHUB SECRETS

SECRET	PURPOSE
`FIREBASE_SERVICE_ACCOUNT`	GCP service account JSON
`GROQ_API_KEY`	Groq LLM for summarization
`HF_API_KEY`	HuggingFace for Kokoro TTS
`GHUB_PAT`	GitHub PAT for issue creation

13. Security Architecture

LAYER	MECHANISM
Authentication	Firebase Auth (Google OAuth 2.0). All Cloud Functions check `request.auth`.
Authorization	Firestore rules: doc ownership (`auth.uid == userId`). Premium only via admin SDK.
Input sanitization	`sanitize-html` with tag + CSS property whitelist. XSS prevention.
Webhook verification	HMAC-SHA256 + `crypto.timingSafeEqual` (timing-attack safe).
Secret management	GCP Secret Manager via `defineSecret()`. Synced from GitHub Secrets in CI.
URL validation	Protocol whitelist (`http:` , `https:` only). Domain-specific handling.
Content security	Safe CSS properties only. Links: `target="_blank" rel="noopener noreferrer"`.

14. Scaling & Cost Analysis

~$0.002	~200 DAU	~100 DAU	#1
COST PER USER/MONTH	GROQ FREE TIER LIMIT	HUGGINGFACE FREE TIER	PROFITABLE FROM SUBSCRIBER

Scaling Bottlenecks

COMPONENT	CURRENT LIMIT	BREAKS AT	SOLUTION
Groq free tier	~30 req/min	~200 DAU	Groq paid ($0.05/M tokens)
HuggingFace free	~100 req/hr	~50-100 DAU	Self-host Kokoro or HF Pro ($9/mo)
Firestore reads	50K/day free	~500 DAU	Blaze plan ($0.06/100K reads)
Client-side sorting	500 docs max	Premium heavy users	Server-side pagination + indexes

15. Monitoring & Observability

SIGNAL	TOOL	LOCATION
Function logs	Cloud Logging (`logger.*`)	Firebase Console → Functions → Logs
Error tracking	Firestore `/errors` collection	Written on summarization failures
Analytics	Firebase Analytics (`G-039HM57SZ4`)	GA4 dashboard
Deployment	GitHub Actions	Two workflows: hosting + functions
Support tickets	Email + Firestore	Admin email on ticket creation
Payments	Paddle Dashboard + Logging	paddleWebhook logs all events